

Отчет по лабораторной 2 «Использование готовых моделей pytorch для классификации»

```
[8]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

!0]: # =====
# ПАРАМЕТРЫ
# =====
BATCH_SIZE = 4
IMG_SIZE = 100
EPOCHS = 100

# сколько эпох ждём улучшения перед остановкой
PATIENCE = 50

!1]: import torchvision.models as models

!2]: # Загружаем ResNet-18 с базовыми весами
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

Рисунок 1 — Импорт библиотек и модели

```
[23]: import torch.nn as nn

# Замораживаем все основные слои (features)
for param in model.parameters():
    param.requires_grad = False

# Заменяем слой классификатора для нашего датасета (например, 10 классов)
num_features = model.fc.in_features
#model.fc = nn.Linear(num_features, 10)
model.fc = nn.Linear(num_features, 2)

[24]: import torch.optim as optim

criterion = nn.CrossEntropyLoss()
# Оптимизируем только параметры последнего слоя
optimizer = optim.SGD(model.fc.parameters(), lr=0.0007, momentum=0.9)

[25]: # =====
# ПРЕОБРАЗОВАНИЯ
# =====
# приводим все картинки к одному размеру + нормализация
transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.5,0.5,0.5], [0.5,0.5,0.5])
])
```

Рисунок 2 — Установка параметров обучения и преобразований

```
[26]: # =====
# ДАННЫЕ
# =====
#train_data = datasets.ImageFolder('train1', transform=transform)
train_data=datasets.ImageFolder(root='train1', transform=transform)
#val_data = datasets.ImageFolder('val1', transform=transform)
val_data=datasets.ImageFolder(root='val1', transform=transform)

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE)
```

Рисунок 3 — Загрузка датасетов

```
[27]: model.train() # Переводим модель в режим обучения

for epoch in range(EPOCHS): # Цикл по эпохам
    running_loss = 0.0
    for inputs, labels in train_loader: # data loading
        optimizer.zero_grad() # Очищаем старые градиенты

        outputs = model(inputs) # Прямой проход (Forward)
        loss = criterion(outputs, labels) # Вычисление функции потерь

        loss.backward() # Обратное распространение ошибки (Backward)
        optimizer.step() # Обновление весов

        running_loss += loss.item()
    # ---- VALIDATION ----
    # проверяем качество на новых данных
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            outputs = model(images)

            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
# ---- Вывод ----
print(f"Epoch {epoch+1}")
print(f"Loss: {running_loss:.4f}")
print(f"Accuracy: {accuracy:.2f}%")
print("-----")
print("Дообучение завершено!")
# 'model' — ваша обученная нейросеть
torch.save(model.state_dict(), 'fine_tuned_weights.pth')
```

Epoch 1

Рисунок 4- Обучение модели и сохранение весов

```
Accuracy: 81.69%
-----
Epoch 89
Loss: 0.5380
Accuracy: 81.69%
-----
Epoch 90
Loss: 0.5302
Accuracy: 80.28%
-----
Epoch 91
Loss: 0.5267
Accuracy: 81.69%
-----
Epoch 92
Loss: 0.5202
Accuracy: 80.28%
-----
Epoch 93
Loss: 0.5143
Accuracy: 81.69%
-----
Epoch 94
Loss: 0.5067
Accuracy: 81.69%
-----
Epoch 95
Loss: 0.5051
Accuracy: 81.69%
-----
Epoch 96
Loss: 0.5098
Accuracy: 81.69%
-----
Epoch 97
Loss: 0.4909
Accuracy: 81.69%
-----
Epoch 98
Loss: 0.4870
Accuracy: 81.69%
-----
```

Рисунок 5 — График обучения

```
[28]: # =====
# ТЕСТ НА ПАПКЕ testbl
# =====
```

```
import os
from PIL import Image
import torch.nn.functional as F
print("\n=== Проверка на тестовых изображениях ===")
```

=== Проверка на тестовых изображениях ===

```
[29]: #test
model = models.resnet18()
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 2)
state_dict = torch.load('fine_tuned_weights.pth', map_location="cpu")

# 4. Load the weights into the model
model.load_state_dict(state_dict)
test_dir = "test"
classes = train_data.classes
```

```
/tmp/ipykernel_1191/569688689.py:5: FutureWarning: You are using `torch.load` with `weights_only=False`
state_dict = torch.load('fine_tuned_weights.pth', map_location='cpu',
Rисунок 6- загрузка модели для тестирования
```

```
[30]: for file in os.listdir(test_dir):

    if file.endswith((".jpg", ".png", ".jpeg")):

        path = os.path.join(test_dir, file)

        img = Image.open(path).convert("RGB")
        img = transform(img)
        img = img.unsqueeze(0)

        with torch.no_grad():
            output = model(img)
            probs = F.softmax(output, dim=1)

            confidence, predicted = torch.max(probs, 1)

            print(f"\nФайл: {file}")
            print("Предсказание:", classes[predicted.item()])
            print("Уверенность:", round(confidence.item(), 3))
```

Файл: english_cocker_spaniel_121_jpg.rf.db065f7b515f83644b2fcb7525073d0b.jpg

Предсказание: dog

Уверенность: 0.979

Файл: yorkshire_terrier_106_jpg.rf.d6f3d2d3861cfdb0b43daab736c5c3d7.jpg

Предсказание: dog

Уверенность: 0.977

Файл: Persian_105_jpg.rf.ea4d455c191a11efb7d07b377717618a.jpg

Предсказание: dog

Уверенность: 0.968

Файл: boxer_12_jpg.rf.e900c37bda862f16a7a42b9fd21fa8e3.jpg

Предсказание: dog

Уверенность: 0.979

Файл: english_setter_165_img.rf.0ca2ff5a48702a310cfd020b0601c718.jpg

Рисунок 6 — Результат тестирования

Датасет использовался тот же что и в 1 лабораторной.